EECS 348 Assignment 3: GenAI Code Analysis

CEO Email Prioritizer (MaxHeap Priority Queue)

Nicolas Villalobos

October 1, 2026

a. GenAI Programs Used

1. **Mistral Le Chat** (Mistral AI)
2. **Claude** (Anthropic)

b. How I Accessed Them

- **Mistral Le Chat:** web interface at https://chat.mistral.ai in my browser, free account, new chat with no prior messages.
- **Claude:** web interface at https://claude.ai in my browser, new chat with no prior messages.

Both were given the identical prompt below as the first and only message, and the raw responses were copied before any edits.

c. Prompt Used (identical for both)

```
Write a C++ program using objects (classes), not standalone functions, that prioritizes emails for a busy
     company CEO.

Requirements:
- Use a MaxHeap to implement a priority queue. Implement the MaxHeap yourself using a list-based (array/
     vector) implementation. Do not use any pre-existing heap modules (no std::priority_queue, no std::
     make_heap or related functions).
- Use that MaxHeap to handle all email prioritizing.

Input: read commands from a test file. The file begins with emails, followed by commands. Commands:

EMAIL <sender category>,<subject line>,<date>
  - EMAIL is followed by a space; the remaining fields are comma-delimited.
  - <sender category> is one of: Boss, Subordinate, Peer, ImportantPerson, OtherPerson
  - Priority order (highest first): Boss, Subordinate, Peer, ImportantPerson, OtherPerson
  - If multiple emails have the same sender category, the newest email (by date) is read first.
  - <subject line> may contain spaces but not commas.
  - <date> is in the format MM-DD-YYYY.

NEXT
  - Display the highest-priority email in this format:
    Next email:
    Sender: <sender category>
    Subject: <subject line>
    Date: <date>
```

```
READ
  - The CEO has read the highest-priority email; remove it from the queue.

COUNT
  - Display: There are <n> emails to read.

The program must handle: NEXT or READ when there are no emails, two NEXTs in a row without a READ (same
    output both times), and two READs in a row without a NEXT (the highest-priority email is removed
    without being displayed).

Sample test file:
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT

Expected output:
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.
```

d. Generated Code

d.1 Mistral Le Chat (raw)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <unordered_map>

// ----------------------------------------------------------------------------
// Email: represents a single email in the CEO's inbox.
// Knows its own sender category, subject line, and date,
// and can compare its priority against another email.
// ----------------------------------------------------------------------------
class Email {
public:
    Email() : category_(""), subject_(""), date_("") {}

    Email(const std::string& category,
          const std::string& subject,
          const std::string& date)
        : category_(category), subject_(subject), date_(date) {}

    // Higher rank = higher priority. Boss first, OtherPerson last.
    static int categoryRank(const std::string& category) {
        static const std::unordered_map<std::string, int> ranks = {
            {"Boss", 5},
            {"Subordinate", 4},
            {"Peer", 3},
            {"ImportantPerson", 2},
            {"OtherPerson", 1}
        };
        auto it = ranks.find(category);
        return (it != ranks.end()) ? it->second : 0;
```

```
    }

    // Date is MM-DD-YYYY; convert to a single sortable number YYYYMMDD
    // so newer dates compare as larger.
    static long dateToNumber(const std::string& date) {
        int mm = 0, dd = 0, yyyy = 0;
        char sep1 = 0, sep2 = 0;
        std::istringstream iss(date);
        if (iss >> mm >> sep1 >> dd >> sep2 >> yyyy) {
            return static_cast<long>(yyyy) * 10000L
                 + static_cast<long>(mm) * 100L
                 + static_cast<long>(dd);
        }
        return 0; // unparseable date sorts oldest
    }

    // True if this email has strictly higher priority than `other`:
    // first by sender category, then (on ties) the newest email first.
    bool higherPriorityThan(const Email& other) const {
        int myRank    = categoryRank(category_);
        int otherRank = categoryRank(other.category_);
        if (myRank != otherRank) return myRank > otherRank;
        return dateToNumber(date_) > dateToNumber(other.date_);
    }

    const std::string& category() const { return category_; }
    const std::string& subject()  const { return subject_; }
    const std::string& date()     const { return date_; }

private:
    std::string category_;
    std::string subject_;
    std::string date_;
};

// ---------------------------------------------------------------------------
// MaxHeap: array-based (vector) max-heap of Emails.
// Implemented from scratch (no std::priority_queue, no make_heap).
// The "largest" element is the highest-priority email.
// ---------------------------------------------------------------------------
class MaxHeap {
public:
    MaxHeap() : heap_() {}

    bool empty() const { return heap_.empty(); }
    int  size()  const { return static_cast<int>(heap_.size()); }

    // Insert a new email and restore the heap property upward.
    void push(const Email& email) {
        heap_.push_back(email);
        siftUp(static_cast<int>(heap_.size()) - 1);
    }

    // Return (without removing) the highest-priority email.
    const Email& top() const {
        return heap_.front();
    }

    // Remove the highest-priority email and restore the heap property downward.
    void pop() {
        if (heap_.empty()) return;
        heap_.front() = heap_.back();
        heap_.pop_back();
        if (!heap_.empty()) siftDown(0);
    }

private:
    std::vector<Email> heap_;

    int parent(int i) const { return (i - 1) / 2; }
    int leftChild(int i) const { return 2 * i + 1; }
    int rightChild(int i) const { return 2 * i + 2; }

    void siftUp(int i) {
        while (i > 0 && heap_[i].higherPriorityThan(heap_[parent(i)])) {
            std::swap(heap_[i], heap_[parent(i)]);
            i = parent(i);
        }
    }

    void siftDown(int i) {
        int n = static_cast<int>(heap_.size());
        while (true) {
            int largest = i;
            int left  = leftChild(i);
            int right = rightChild(i);
            if (left  < n && heap_[left].higherPriorityThan(heap_[largest]))
                largest = left;
            if (right < n && heap_[right].higherPriorityThan(heap_[largest]))
                largest = right;
            if (largest == i) break;
            std::swap(heap_[i], heap_[largest]);
```

```
            i = largest;
        }
    }
};

// ---------------------------------------------------------------------------
// EmailPrioritizer: the CEO's email manager.
// Owns the MaxHeap and processes all commands from the test file.
// ---------------------------------------------------------------------------
class EmailPrioritizer {
public:
    explicit EmailPrioritizer(std::ostream& out = std::cout) : heap_(), out_(out) {}

    void addEmail(const std::string& category,
                  const std::string& subject,
                  const std::string& date) {
        heap_.push(Email(category, subject, date));
    }

    void nextEmail() {
        if (heap_.empty()) {
            out_ << "No emails to read.\n";
            return;
        }
        const Email& email = heap_.top();
        out_ << "Next email:\n"
             << "Sender: " << email.category() << "\n"
             << "Subject: " << email.subject() << "\n"
             << "Date: " << email.date() << "\n";
    }

    void readEmail() {
        if (heap_.empty()) {
            out_ << "No emails to read.\n";
            return;
        }
        heap_.pop();
    }

    void countEmails() const {
        out_ << "There are " << heap_.size() << " emails to read.\n";
    }

    // Read the whole test file and execute each command.
    // Returns false if the file cannot be opened.
    bool processFile(const std::string& filename) {
        std::ifstream in(filename);
        if (!in.is_open()) return false;

        std::string line;
        while (std::getline(in, line)) {
            // Trim trailing carriage returns (Windows line endings).
            if (!line.empty() && line.back() == '\r') line.pop_back();

            if (line.rfind("EMAIL ", 0) == 0) {
                std::string body = line.substr(6);
                std::string category, subject, date;
                if (parseEmail(body, category, subject, date)) {
                    addEmail(category, subject, date);
                }
            } else if (line == "NEXT") {
                nextEmail();
            } else if (line == "READ") {
                readEmail();
            } else if (line == "COUNT") {
                countEmails();
            }
            // Blank or unknown lines are ignored.
        }
        return true;
    }

private:
    MaxHeap heap_;
    std::ostream& out_;

    // Split "Category,Subject,Date" into its three comma-delimited fields.
    // The subject may contain spaces but never commas.
    static bool parseEmail(const std::string& body,
                           std::string& category,
                           std::string& subject,
                           std::string& date) {
        std::string::size_type first = body.find(',');
        if (first == std::string::npos) return false;
        std::string::size_type second = body.find(',', first + 1);
        if (second == std::string::npos) return false;

        category = body.substr(0, first);
        subject  = body.substr(first + 1, second - first - 1);
        date     = body.substr(second + 1);
        return !category.empty() && !date.empty();
    }
```

```
};

// ---------------------------------------------------------------------------
// main: run the CEO email prioritizer on a test file.
// Usage: ./ceo_emails <test-file>   (defaults to "test.txt" if omitted)
// ---------------------------------------------------------------------------
int main(int argc, char* argv[]) {
    std::string filename = (argc > 1) ? argv[1] : "test.txt";

    EmailPrioritizer prioritizer;
    if (!prioritizer.processFile(filename)) {
        std::cerr << "Error: could not open file \"" << filename << "\".\n";
        return 1;
    }
    return 0;
}
```

d.2 Claude (raw)

```
// Email prioritizer for a busy CEO.
// Uses a hand-written, vector-based MaxHeap as the priority queue.

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <stdexcept>

// ---------------------------------------------------------------------------
// Email: one message plus the logic for comparing priority between messages.
// ---------------------------------------------------------------------------
class Email {
public:
    Email() : rank(0), sortableDate(0) {}

    Email(const std::string& category, const std::string& subject, const std::string& date)
        : category(category), subject(subject), date(date),
          rank(categoryRank(category)), sortableDate(toSortableDate(date)) {}

    const std::string& getCategory() const { return category; }
    const std::string& getSubject()  const { return subject; }
    const std::string& getDate()     const { return date; }

    // True if this email should be read before the other one.
    bool hasHigherPriorityThan(const Email& other) const {
        if (rank != other.rank) return rank > other.rank;
        return sortableDate > other.sortableDate;   // newer first on ties
    }

    void display(std::ostream& out) const {
        out << "Next email:\n"
            << "Sender: "  << category << "\n"
            << "Subject: " << subject  << "\n"
            << "Date: "    << date     << "\n";
    }

    static bool isValidCategory(const std::string& c) { return categoryRank(c) > 0; }

private:
    std::string category;
    std::string subject;
    std::string date;
    int rank;          // higher = more important
    long sortableDate; // YYYYMMDD so plain integer comparison orders by date

    static int categoryRank(const std::string& c) {
        if (c == "Boss")            return 5;
        if (c == "Subordinate")     return 4;
        if (c == "Peer")            return 3;
        if (c == "ImportantPerson") return 2;
        if (c == "OtherPerson")     return 1;
        return 0;
    }

    // "MM-DD-YYYY" -> YYYYMMDD
    static long toSortableDate(const std::string& d) {
        if (d.size() != 10 || d[2] != '-' || d[5] != '-')
            throw std::invalid_argument("Bad date format: " + d);
        int month = std::stoi(d.substr(0, 2));
        int day   = std::stoi(d.substr(3, 2));
        int year  = std::stoi(d.substr(6, 4));
        return static_cast<long>(year) * 10000 + month * 100 + day;
    }
};

// ---------------------------------------------------------------------------
// MaxHeap: array (vector) based binary heap of Emails.
```

```
// Parent of i is (i-1)/2; children are 2i+1 and 2i+2.
// ---------------------------------------------------------------------------
class MaxHeap {
public:
    bool isEmpty() const { return data.empty(); }
    std::size_t size() const { return data.size(); }

    void insert(const Email& e) {
        data.push_back(e);
        siftUp(data.size() - 1);
    }

    const Email& peekMax() const {
        if (isEmpty()) throw std::out_of_range("peekMax on empty heap");
        return data[0];
    }

    Email removeMax() {
        if (isEmpty()) throw std::out_of_range("removeMax on empty heap");
        Email top = data[0];
        data[0] = data.back();
        data.pop_back();
        if (!data.empty()) siftDown(0);
        return top;
    }

private:
    std::vector<Email> data;

    void swapAt(std::size_t a, std::size_t b) {
        Email tmp = data[a];
        data[a] = data[b];
        data[b] = tmp;
    }

    void siftUp(std::size_t i) {
        while (i > 0) {
            std::size_t parent = (i - 1) / 2;
            if (!data[i].hasHigherPriorityThan(data[parent])) break;
            swapAt(i, parent);
            i = parent;
        }
    }

    void siftDown(std::size_t i) {
        std::size_t n = data.size();
        while (true) {
            std::size_t left = 2 * i + 1;
            std::size_t right = 2 * i + 2;
            std::size_t largest = i;
            if (left < n && data[left].hasHigherPriorityThan(data[largest]))   largest = left;
            if (right < n && data[right].hasHigherPriorityThan(data[largest])) largest = right;
            if (largest == i) break;
            swapAt(i, largest);
            i = largest;
        }
    }
};

// ---------------------------------------------------------------------------
// EmailManager: reads the command file and drives the heap.
// ---------------------------------------------------------------------------
class EmailManager {
public:
    explicit EmailManager(std::ostream& out) : out(out) {}

    void processFile(const std::string& filename) {
        std::ifstream in(filename);
        if (!in) {
            out << "Error: could not open file \"" << filename << "\".\n";
            return;
        }
        std::string line;
        while (std::getline(in, line)) {
            processLine(trim(line));
        }
    }

private:
    MaxHeap queue;
    std::ostream& out;

    void processLine(const std::string& line) {
        if (line.empty()) return;

        if (line.rfind("EMAIL ", 0) == 0)  handleEmail(line.substr(6));
        else if (line == "NEXT")           handleNext();
        else if (line == "READ")           handleRead();
        else if (line == "COUNT")          handleCount();
        else out << "Unknown command: " << line << "\n";
    }

```

```
    void handleEmail(const std::string& fields) {
        // Split into exactly three comma-delimited fields.
        std::size_t first = fields.find(',');
        std::size_t last  = fields.rfind(',');
        if (first == std::string::npos || first == last) {
            out << "Malformed EMAIL line: " << fields << "\n";
            return;
        }
        std::string category = trim(fields.substr(0, first));
        std::string subject  = trim(fields.substr(first + 1, last - first - 1));
        std::string date     = trim(fields.substr(last + 1));

        if (!Email::isValidCategory(category)) {
            out << "Unknown sender category: " << category << "\n";
            return;
        }
        try {
            queue.insert(Email(category, subject, date));
        } catch (const std::exception& e) {
            out << "Malformed EMAIL line (" << e.what() << ")\n";
        }
    }

    // NEXT only looks; repeated NEXTs show the same email.
    void handleNext() {
        if (queue.isEmpty()) {
            out << "There are no emails to read.\n";
            return;
        }
        queue.peekMax().display(out);
    }

    // READ removes the top email whether or not it was displayed.
    void handleRead() {
        if (queue.isEmpty()) {
            out << "There are no emails to read.\n";
            return;
        }
        queue.removeMax();
    }

    void handleCount() {
        out << "There are " << queue.size() << " emails to read.\n";
    }

    static std::string trim(const std::string& s) {
        const char* ws = " \t\r\n";
        std::size_t start = s.find_first_not_of(ws);
        if (start == std::string::npos) return "";
        std::size_t end = s.find_last_not_of(ws);
        return s.substr(start, end - start + 1);
    }
};

int main(int argc, char* argv[]) {
    std::string filename;
    if (argc > 1) {
        filename = argv[1];
    } else {
        std::cout << "Enter test file name: ";
        std::getline(std::cin, filename);
    }
    EmailManager manager(std::cout);
    manager.processFile(filename);
    return 0;
}
```

e. Analysis

I compiled both with `g++ -std=c++17 -O2` and ran them on the sample file, an edge-case file, and a stress file of 500,000 emails followed by 500,000 NEXT/READ pairs.

	Mistral	Claude
Sample file: correct emails in correct order	Yes	Yes
Matches assignment display format (blank lines, indentation)	No	No
Double NEXT / double READ / empty inbox	Yes	Yes
Trims spaces after commas	No	Yes
Rejects unknown category or bad date	No	Yes
Stdout clean and error exit code on missing file	Yes	No
Same category and date: deterministic order	No	No
Time, 500k emails (best of 3)	4.91 s	1.37 s
Peak memory, 500k emails	72.0 MB	81.3 MB

Correctness

Both programs read the emails in the correct order and handle repeated NEXT (peek without removing) and repeated READ (remove without printing). Both convert `MM-DD-YYYY` into the number `YYYYMMDD`, so `01-03-2025` correctly comes before `12-20-2024`; comparing the dates as strings would get this wrong. Neither prints the blank lines and indented fields shown in the assignment's expected output.

Mistral does no input checking. An unknown category is queued with rank 0, an unreadable date becomes 0 and sorts as the oldest, and a space after a comma is printed as part of the subject and date. It also prints a message on READ of an empty inbox even though READ never prints otherwise.

Claude validates categories and dates and trims whitespace. Its bugs are in `main`: with no argument, it prints `Enter test file name:` to stdout, which would mix into graded output, and a missing file still returns exit code 0. Its date check also does not confirm that the characters are digits.

Neither program decides which email comes first when two share both category and date, so the order depends on how the heap happened to be built.

Execution time

Both use an array-based binary heap: insert and remove are $O(\log n)$, peek and count are $O(1)$. The difference is the cost of each comparison. Mistral looks up the category in a hash map and re-parses the date with an `istringstream` every time two emails are compared, and each insert or remove does about $2\log_2 n$ comparisons. Claude computes the rank and date once when the email is created, so a comparison is two integer checks. This is why Claude is 3.6× faster. Claude's `swapAt` does make three full copies of an email (three strings each) per swap, and `removeMax` returns a copy that is never used.

Space complexity

Both are $O(n)$, one vector slot per email, and both sift loops are iterative so there is no recursion stack. Claude stores an extra `int` and `long` per email, which is why it used about 13% more memory. That is a reasonable trade for the speedup.

Maintainability

Both split the work into three classes (email, heap, command manager) with private data and `const` methods. Claude keeps all priority rules in the `Email` class and reports bad input, which makes problems easy to find. Mistral hides errors by quietly assigning default values. Neither has the prologue, block comments, or line comments the course rubric requires.

f. Selection and Justification

I chose **Claude** as the base. It was 3.6× faster on the stress test (1.37 s vs. 4.91 s), it handles bad input that Mistral silently accepts, and its remaining bugs are small and isolated in `main` and the output format. Its only cost is about 13% more memory, and both are $O(n)$.

g. Improvements Made

Correctness

- Output now matches the assignment: a blank line between outputs and tab-indented Sender, Subject, and Date lines.
- The file-name prompt and error messages go to stderr, so only graded output reaches stdout. The program returns 1 if the file cannot be opened.
- READ on an empty inbox prints nothing, since READ never displays anything.
- Each email gets a sequence number from its position in the file; if category and date tie, the later email is read first.
- Dates must be digits with valid month (1–12) and day (1–31).

I verified the output against the sample file and against a brute-force sorted reference on 200 random test files; all matched.

Execution time

I replaced the copying `swapAt` with `std::swap`, which moves strings instead of copying them, and made `removeMax` return nothing so it no longer copies the removed email. Big-O is unchanged, but the stress test dropped from 1.37 s to 0.99 s (about 28% faster).

Space complexity

Removing those copies eliminates temporary string allocations during each sift. Each email gains one 8-byte sequence number for the tie-break, so space is still $O(n)$ (83.4 MB peak vs. 81.3 MB).

Maintainability

I added the full prologue (program name, description, inputs, output, collaborators + other sources, author, dates, revisions), a block comment on each class stating who wrote it, and a comment on every line. Errors and normal output now use separate streams, which makes the manager class easier to test.